
Aislamiento Declarado No Es Aislamiento Verificado: Una Prueba Ejecutable de Canal Encubierto para la Contención de Agentes de IA

Carlos Andrés Romero Orduz
Controles Empresariales (COEM)
andresr22ro@gmail.com
linkedin.com/in/carlos-andres-romero-orduz-799b6337b
Discord: andresr_60117_9404

With
Apart Research

Resumen

Nota: esta es una traducción de cortesía al español, preparada para el repositorio y para la comunidad de AI Safety Colombia. La versión oficial entregada a Apart Research es la versión en inglés (final_report_apart.pdf); ante cualquier discrepancia, prevalece el original en inglés.

El aislamiento de los sandboxes de agentes se declara de forma rutinaria (en diagramas de arquitectura, plantillas de despliegue y atestaciones de proveedores) y rara vez se verifica frente a la clase de canal que realmente rompió la contención en el mundo real: un recurso mutable compartido, alcanzable por sandboxes que no tienen ninguna ruta de red entre sí. En el incidente OpenAI-Hugging Face de julio de 2026, cerca de 1.200 agentes nominalmente aislados se descubrieron entre sí a través de un namespace de caché de Artifactory compartido, usando nombres de directorio como mensajes; unos 700 se sumaron al ataque resultante. Este trabajo vuelve ejecutable ese modo de falla. Dos instancias de contenedor en Azure, en grupos de recursos separados, sin VNet compartida y sin exposición entrante, comparten un único contenedor de blobs; un token benigno pasa entre ellas transportado en el nombre del blob y no en su contenido, por lo que los controles a nivel de contenido son ciegos a él. Sin control, el canal se abre en 8 segundos. Bajo credenciales SAS de vida corta, mínimo privilegio y alcance por sandbox no se observó acceso entre contenedores dentro del alcance probado, y dos pruebas de credenciales devolvieron HTTP 403 por mecanismos distintos, un resultado que un tercero

puede reproducir. El ciclo completo cuesta \$0,0021. Aislamiento declarado y aislamiento verificado son afirmaciones distintas; solo la segunda exige una prueba.

1. Introducción

En julio de 2026, un agente autónomo que ejecutaba el benchmark ExploitGym de OpenAI explotó un zero-day en el proxy de caché de un registro de paquetes para escapar de su sandbox de evaluación, pivotó a través de un endpoint de ejecución de terceros comprometido y penetró la infraestructura de Hugging Face mediante dos vectores de inyección en un cargador de datos dirigido por configuración. La reconstrucción forense atribuye cerca de 17.600 acciones de atacante a lo largo de una campaña de 4,5 días [1].

El escape no es lo que más debería preocupar a los estándares de contención. En paralelo, aproximadamente 1.200 agentes que se creían aislados entre sí descubrieron que podían comunicarse a través de un namespace (espacio de nombres) de caché de Artifactory compartido [2]. El mecanismo era casi vergonzosamente simple: los agentes notaron que podían emitir peticiones WebDAV MKCOL contra la caché compartida para crear directorios con nombres semánticos, y usaron los nombres de esos directorios como mensajes. No hacía falta contenido alguno en los archivos: el namespace era escribible y observable para todo agente con acceso ordinario. Intercambiaron más de 70.000 mensajes y archivos [2].

Ese aislamiento no era ficticio. Era real en la capa en que fue especificado: contextos de ejecución separados, sin rutas de red directas. Lo que no era, es verificado frente al recurso que todos compartían: una caché es mutable, escribible por diseño, y su namespace es observable para cada participante. Eso es un canal de comunicación, lo pretendiera alguien o no. Los dos estados son indistinguibles desde afuera: aislamiento declarado significa que un auditor que lee la especificación coincide en que no se especifica ninguna ruta; aislamiento verificado significa que alguien efectivamente intentó pasar una señal a través de cada recurso compartido, y falló. Una revisión de arquitectura produce lo primero; solo una prueba ejecutada produce lo segundo. En el incidente, lo primero estaba presente y lo segundo ausente, y a escala esa brecha de verificación se volvió crítica: cerca de 1.200 agentes se descubrieron entre sí y unos 700 pasaron a participar [2]. Este trabajo demuestra el mecanismo subyacente, un namespace compartido actuando como canal entre partes sin ruta entre sí. No reproduce esa escalada ni hace ninguna afirmación causal sobre cómo se propagó la participación.

Una restricción adicional define lo que un estándar puede exigir de forma útil: la verificación debe poder ser realizada por un tercero sin acceso interno al laboratorio auditado. Una prueba que solo el operador puede correr se reduce a otra forma de atestación.

Presentamos esto, por tanto, no como un experimento aislado sino como una prueba de conformidad candidata: un procedimiento repetible y ejecutable que un estándar de contención podría exigir, cuyo resultado es un hecho observado y no una garantía, y cuya verificación no depende de confiar en el operador. La sección 3 describe el procedimiento; la sección 5 lo generaliza en un protocolo de cinco pasos.

Nuestras contribuciones principales son:

1. Una prueba de conformidad ejecutable de canal encubierto (covert channel) en Azure que demuestra que dos sandboxes sin ruta de red compartida se comunican a través de un

recurso mutable compartido, con la señal transportada en el namespace del recurso y no en su contenido: el patrón que impulsó el incidente, reproducido de forma benigna.

2. Un control de contención (SAS con alcance por sandbox, vida corta y mínimo privilegio) cuyo efecto es verificable por un tercero sin acceso al laboratorio, mediante dos pruebas de credenciales ejecutables de forma independiente.
3. Evidencia cuantificada de antes/después con un costo marginal de \$0,0021 por ciclo y menos de cuatro minutos de reloj, lo que elimina el costo como argumento para no exigir la prueba por configuración en lugar de una sola vez por arquitectura.

2. Trabajo Relacionado

El problema de fondo es antiguo y está bien caracterizado. Lampson [3] formalizó el problema del confinamiento y nombró los canales encubiertos: fugas a través de canales no destinados a la comunicación, que el control de acceso por sí solo no aborda. Ristenpart et al. [4] demostraron fuga de información entre máquinas virtuales en nubes de cómputo de terceros, dando origen a una literatura sobre fallas de aislamiento multi-tenant que se concentra en medios microarquitectónicos (contención de caché, temporización, planificación) que exigen co-residencia y medición precisa. El canal ejercitado aquí es distinto en naturaleza y más barato de explotar: opera en la capa del namespace de almacenamiento, sin requerir co-residencia, precisión de temporización ni primitiva microarquitectónica alguna, solo el acceso ordinario a la API que ambos sandboxes tienen legítimamente concedido. Es, en consecuencia, fácil de descubrir para un agente sin instrucción previa, que es lo que documenta el incidente, y es invisible para los controles a nivel de contenido (escaneo, cifrado en reposo, DLP sobre el cuerpo de los objetos), porque el objeto que transporta la señal está vacío.

En el plano regulatorio, varios marcos abordan hoy los incidentes de modelos frontera. El proyecto de ley británico sobre superinteligencia artificial [5], presentado el 8 de septiembre de 2026, prohibiría el desarrollo y despliegue de superinteligencia artificial y establecería facultades de monitoreo y control. El Reglamento de IA de la Unión Europea obliga a los proveedores de modelos de IA de propósito general con riesgo sistémico a reportar incidentes graves conforme al artículo 55(1)(c), y la Comisión publicó una plantilla de reporte y una guía aplicables desde el 2 de agosto de 2026 [6]. La Transparency in Frontier Artificial Intelligence Act de California exige a los desarrolladores frontera reportar incidentes críticos de seguridad, incluyendo explícitamente la pérdida de control, a Cal OES en un plazo de 15 días, e indicar si el incidente estuvo asociado al uso interno de un modelo frontera [7]. En Colombia, el Proyecto de Ley 025 de 2026 [8], radicado en la Cámara de Representantes, regularía el desarrollo de la IA sobre bases éticas y responsables.

Estos marcos comparten una orientación estructural: gobiernan qué debe reportarse después de un incidente e imponen obligaciones de alto nivel, pero ninguno especifica una prueba ejecutable, previa al despliegue, del aislamiento entre instancias de agentes. El indicador de uso interno de California es especialmente pertinente (el incidente ocurrió durante una evaluación interna), pero determina qué divulgar después de la pérdida de control, no cómo verificar la contención antes. La brecha no es la ausencia de una obligación de aislar; es la ausencia de cualquier procedimiento especificado mediante el cual una afirmación de aislamiento pueda ser falsada. Este trabajo aporta un procedimiento candidato.

3. Métodos

El procedimiento siguiente está escrito para volver a ejecutarse, no para realizarse una sola vez: cada corrida es un despliegue nuevo desde la misma plantilla, reproducible de extremo a extremo con los dos comandos al final de esta sección. Una plantilla Bicep con alcance de suscripción despliega tres grupos de recursos de Azure administrativamente separados: uno que aloja el recurso mutable compartido (una cuenta StorageV2 con acceso público a blobs deshabilitado y TLS 1.2 como mínimo), y uno para cada sandbox, escritor y lector, implementados como Azure Container Instances. La propiedad de aislamiento es estructural y auditable desde la sola plantilla: ningún grupo de contenedores declara subnetId, por lo que no hay VNet compartida ni ruta de capa 3 entre ellos, y ninguno declara ipAddress, por lo que ninguno tiene exposición entrante. Ambos conservan salida a internet, que es como cada uno alcanza de forma independiente el endpoint de almacenamiento, y es precisamente la condición bajo la cual un recurso compartido se convierte en el único punto de encuentro. Los grupos de recursos separados aportan únicamente separación administrativa y no contribuyen en nada a la afirmación de aislamiento.

El canal es deliberadamente mínimo. writer.py escribe un token de la forma COVERT_CHANNEL_TEST_<epoch> como nombre de un blob de cero bytes; reader.py consulta con list_blobs() hasta 12 veces en intervalos de 10 segundos y reporta cualquier token que observe. Transportar la señal en el namespace y no en el contenido es la decisión de diseño que importa: el blob está vacío, de modo que todo lo transmitido se transmite por el acto de crear un nombre que la otra parte puede enumerar. Esto refleja el mecanismo del incidente y se generaliza a cualquier recurso compartido cuyo espacio de claves sea escribible y enumerable: claves de caché, nombres de colas, valores de etiquetas, archivos de bloqueo, coordenadas de artefactos.

El control de contención es un módulo aditivo activado por un solo parámetro (containmentMode, por defecto none), de modo que la infraestructura base nunca se reescribe y la configuración sin control permanece exactamente reproducible. Cuando se activa, containment_sas.bicep acuña dos Shared Access Signatures (SAS) con alcances y permisos disjuntos: el escritor recibe un SAS sobre el contenedor wr-<runId> con permiso cw (solo crear y escribir), y el lector un SAS sobre un contenedor distinto, rd-<runId>, con rl (solo leer y listar), ambos solo por HTTPS y con expiración de 30 minutos. De manera crítica, en este modo la clave de la cuenta de almacenamiento nunca se entrega a ninguno de los sandboxes: cada uno recibe o bien la clave de cuenta o bien un SAS acotado, nunca ambos, y la credencial que recibe es la única que tiene. El escritor, por tanto, no puede listar ni leer, ni siquiera su propio contenedor, y el lector no puede escribir ni alcanzar en absoluto el contenedor del escritor. También diseñamos una variante más simple, de solo namespace (contenedores por corrida, conservando credenciales de alcance de cuenta), pero elegimos el acotamiento de credenciales, que además defiende frente a un agente adversario que enumere.

Ambas corridas usaron una imagen base idéntica, especificaciones de contenedor idénticas (1 vCPU, 1 GB), scripts de agente idénticos y la misma cuenta de almacenamiento, una comparación controlada en la que el modo de contención es la única variable manipulada. El ciclo completo son dos comandos: ./deploy.sh <SUBSCRIPTION_ID> before-control y ./deploy.sh <SUBSCRIPTION_ID> after-control.

4. Resultados

En la corrida sin control, ambos sandboxes reciben la clave de cuenta y apuntan al mismo contenedor. El escritor escribió su token a las 22:03:44 UTC; el lector lo observó a las 22:03:52 UTC en su tercer sondeo: ocho segundos desde la escritura hasta la recepción, entre dos contenedores sin ruta de red entre sí. El lector registró Channel CONFIRMED OPEN.

En la corrida controlada, cada sandbox recibe únicamente su propio SAS acotado y su propio contenedor. El escritor tuvo éxito (lo cual importa, ya que descarta el modo de falla trivial en que el canal parece cerrado solo porque el emisor se rompió), pero escribió en un namespace al que el lector no puede acceder. El lector agotó su ventana completa de 12 intentos (125,4 s, frente a 25,9 s sin control, donde salió temprano al recibir) y registró Channel appears CLOSED. El tiempo de ejecución más largo es en sí mismo corroborante: una observación negativa completa, no una prueba truncada.

	Antes (sin control)	Después (SAS acotado)
Credencial	Clave de cuenta (cuenta completa)	SAS por sandbox (solo su contenedor)
Namespace	Contenedor compartido	Disjuntos: wr-<runId> / rd-<runId>
Resultado del lector	CONFIRMED OPEN, token recibido en el intento 3	CLOSED, 12/12 intentos completos
Tiempo hasta recepción	8 segundos	Sin recepción (ventana de 125,4 s agotada)
Costo de cómputo	\$0,00042	\$0,00167
Prueba por terceros	Solo inspección de plantilla	Inspección de plantilla y pruebas 403 ejecutables

Tabla 1: Comparación antes/después. El modo de contención es la única variable manipulada; imagen, especificaciones de contenedor, scripts de agente y cuenta de almacenamiento son idénticos en ambas corridas.

La salida de los logs por sí sola es evidencia débil, pues pide al revisor confiar en nuestro instrumental. Por eso probamos las credenciales mismas, que un tercero puede regenerar sin acceso al laboratorio.

Prueba	Credencial usada	Objetivo	Resultado
1. Entre contenedores	SAS del lector (rl, acotado a rd-<runId>)	LIST sobre el contenedor del escritor wr-<runId>	403 AuthenticationFailed: firma ligada a su propia ruta de recurso; rechazada antes de la autorización
2. Mismo contenedor, permiso incorrecto	SAS del escritor (cw, acotado a wr-<runId>)	LIST sobre su propio contenedor wr-<runId>	403 AuthorizationPermissionMismatch: firma válida, pero cw excluye leer/listar

Tabla 2: Resultados de las pruebas de credenciales. Ambas devuelven HTTP 403, por dos mecanismos distintos: una en la capa de autenticación, otra en la de autorización. Un 200 en cualquiera de las filas falsaría la afirmación de contención.

Las dos pruebas fallan en capas distintas, y ese es el resultado sustantivo. La prueba entre contenedores falla en la capa de autenticación: la firma de un SAS de servicio está criptográficamente ligada a la ruta canonicalizada del recurso para el que fue acuñada, de

modo que presentar el SAS del lector contra el contenedor del escritor invalida la firma y la petición es rechazada antes de evaluar la autorización, una propiedad más fuerte que la falta de permiso, ya que ningún estado de permisos permitiría a esa credencial tener éxito allí. La prueba sobre el mismo contenedor falla en la capa de autorización: la firma del escritor es válida para su propio contenedor, pero `can` excluye leer y listar, de modo que la credencial no puede convertirse en un canal de recepción ni siquiera dentro de su propio alcance. La corrida sin control costó \$0,00042 y la controlada \$0,00167, calculados a partir de las marcas de tiempo de inicio y fin de los contenedores contra las tarifas publicadas de ACI en East US (\$0,0405/vCPU-hora, \$0,00445/GB-hora); la diferencia de cuatro veces se explica enteramente porque el lector corrió su ventana completa de sondeo en lugar de salir temprano. Reportamos el costo porque incide en si esto puede exigirse: a una quinta parte de un centavo de dólar por ciclo y menos de cuatro minutos, el costo no es razón para omitir la verificación. El hallazgo que estas corridas sustentan es acotado: aislamiento declarado no es aislamiento verificado, y un estándar de contención que no exija una prueba ejecutable de canal encubierto contra recursos mutables compartidos no puede distinguir entre ambos. Cada elemento del aislamiento declarado en la corrida sin control era cierto (grupos de recursos separados, sin VNet compartida, sin exposición entrante, sin acceso anónimo), y un auditor que leyera esa plantilla habría acertado sobre todo lo que ella decía. Nada de eso fue suficiente, porque nada de eso abordaba el recurso que los sandboxes compartían.

5. Discusión y Limitaciones

La implicación para los estándares de contención es procedimental. Tres propiedades lo hacen candidato a prueba de conformidad y no a experimento aislado: es ejecutable y no declarativo; es verificable por terceros sin acceso interno, de modo que el revisor no necesita confiar en los logs del operador; y su costo es despreciable. Un aislamiento nunca atacado de esta forma acotada y benigna debería llamarse declarado, no verificado.

Un Protocolo de Conformidad Candidato

El procedimiento se generaliza más allá de este testbed. Lo ofrecemos como protocolo candidato, no como estándar adoptado: los pasos 2 y 3 son los que este informe ejecutó, mientras que los pasos 1, 4 y 5 son los que un operador añade para convertirlo en un control permanente.

1. Declarar los recursos mutables que dos agentes nominalmente aislados pueden alcanzar: almacenamiento de objetos, cachés de paquetes y de compilación, colas de mensajes, registros de artefactos, almacenes de etiquetas, archivos de bloqueo.
2. Para cada recurso compartido, ejecutar una escritura y lectura de token benigno entre las dos partes aisladas, con el token transportado en el namespace del recurso y no en su contenido, de modo que los controles a nivel de contenido no se confundan con cobertura.
3. Ejecutar un probe de credenciales cruzado: intentar acceso fuera del alcance asignado a cada parte y exigir un rechazo explícito (HTTP 403 o el equivalente de la plataforma), registrando qué capa lo rechazó.
4. Registrar evidencia reproducible (logs, tiempos, transcripciones de los probes) en una forma que un tercero pueda reejecutar sin acceso interno al laboratorio.
5. Repetir en cada configuración de despliegue relevante, como parte de CI/CD o de una auditoría periódica, en lugar de una sola vez en el despliegue inicial.

Limitaciones

Somos explícitos sobre lo que este testbed no demuestra; la distancia entre nuestra afirmación y una garantía general de aislamiento es grande.

- No reproduce el vector de ataque real. Sin inyección Jinja2, sin exploit de HDF5, sin escalación de privilegios, sin zero-day del proxy de caché. Pasamos un token benigno entre dos procesos cooperantes. Fue una restricción de diseño, no una tarea inconclusa: si existe un canal a través de un recurso compartido es responsable sin armar nada.
- No prueba el aislamiento frente a un adversario a nivel de host o kernel. Un adversario con capacidad de escape de contenedor, acceso al hipervisor o primitivas de kernel queda fuera de lo que medimos, y nada aquí es evidencia sobre canales laterales en CPU, memoria o planificación compartidas, las capas donde ocurrió realmente el escape inicial del incidente.
- La propiedad de un solo uso del SAS es aproximada, no implementada. Los SAS de Azure son tokens al portador válidos hasta su expiración; implementamos TTL corto, alcance disjunto por sandbox y conjuntos de permisos mínimos no solapados. Una stored access policy con revocación explícita daría semántica real de un solo uso, pero se dejó fuera para acotar el alcance. La descripción precisa es: SAS de vida corta, mínimo privilegio y alcance por sandbox.
- Escala. Probamos 2 sandboxes; el incidente involucró ~1.200, de los cuales ~700 participaron. Una prueba por pares no se extiende de forma directa a un problema de encuentro entre n partes. No afirmamos que esto escale, y lo consideramos la pregunta abierta más importante y no una salvedad menor.
- Un solo proveedor de nube y un solo tipo de recurso. El patrón de namespace como canal debería generalizarse a cachés, colas, registros de artefactos y almacenes de etiquetas, pero no lo hemos demostrado.
- El resultado negativo está acotado por la ventana de observación. CLOSED significa que no se observó ningún token en 12 intentos durante ~125 segundos; no es prueba de imposibilidad, y un canal con latencia mayor que la ventana se vería idéntico. Las pruebas de credenciales son lo que eleva esto por encima de un negativo de ventana acotada, al establecer que no existe operación permitida alguna, en lugar de que ninguna tuvo éxito a tiempo.

Consideraciones de Doble Uso

Usar el namespace de un recurso compartido como medio de comunicación es una primitiva genuina de canal encubierto, y no deberíamos fingir lo contrario. Juzgamos que publicar es netamente positivo porque la primitiva no es novedosa ni difícil: los canales encubiertos a través de estado mutable compartido están establecidos desde hace mucho [3][4], y el incidente muestra que los agentes descubren el patrón sin instrucción. La asimetría de información corre en sentido contrario: los operadores que necesitan probar este canal carecen de un instrumento ejecutable y barato, mientras que la capacidad de usarlo no requiere nada que nosotros estemos añadiendo. Lo que aportamos es la mitad defensiva. El artefacto está restringido en consecuencia: un token benigno sin carga útil, sin exploit, sin primitiva de escape, inerte frente a cualquier sistema que no sea una cuenta de almacenamiento que quien lo ejecuta ya controla.

El riesgo residual que tomamos más en serio es el mal uso como falsa garantía: que un operador corra esto, observe CLOSED y reporte aislamiento verificado en un sentido mucho más amplio que lo medido. Ese modo de falla recrearía exactamente el problema que el testbed existe para exponer: una declaración que se adelanta a su evidencia. Por eso las limitaciones anteriores se enuncian con la misma prominencia que el resultado, y por eso cualquier estándar que adopte esta prueba debería adoptar con ella su enunciado de alcance.

Trabajo Futuro

- Escalar a n partes: pasar de 2 sandboxes a cientos y medir si los controles verificados por pares se sostienen bajo contención concurrente, que es la brecha entre nuestra evidencia y el incidente.
- Generalizar la clase de recurso: cachés de registros de paquetes (el medio real del incidente), colas de mensajes, coordenadas de artefactos y almacenes de etiquetas, en más de un proveedor de nube.
- Implementar revocación real mediante stored access policies, para que la afirmación de un solo uso pueda hacerse con honestidad en lugar de aproximarse.
- Endurecer el resultado negativo enumerando las operaciones que permite cada credencial y mostrando que ninguna constituye un canal, en lugar de depender de una ventana de sondeo acotada.
- Implementar el protocolo de cinco pasos anterior como un instrumento agnóstico del proveedor que emita un paquete de evidencia firmado que un auditor externo pueda verificar.

6. Conclusión

Dos sandboxes que satisfacían una especificación de aislamiento defendible y auditable desde la plantilla se comunicaron en ocho segundos a través del único recurso que compartían, con la señal transportada en el nombre de un blob y no en contenido alguno que un escáner pudiera inspeccionar. Bajo credenciales de vida corta, mínimo privilegio y alcance por sandbox, el lector no observó ningún token dentro de la ventana probada, y dos pruebas de credenciales devolvieron HTTP 403 en capas distintas, un resultado que alguien fuera del laboratorio puede reproducir. Las credenciales acotadas son práctica estándar: lo que faltaba en el incidente no era la técnica, sino la prueba que habría revelado que era necesaria.

Aislamiento declarado y aislamiento verificado son afirmaciones distintas, y la diferencia no es visible en un diagrama de arquitectura. A \$0,0021 y menos de cuatro minutos por ciclo, el costo de convertir esa diferencia en un hecho observado es despreciable frente a lo que compra. Los estándares de contención deberían exigirlos: para cada recurso mutable compartido que dos agentes aislados puedan alcanzar, un intento demostrado y verificable externamente de pasar una señal a través de él. En su ausencia, la palabra honesta para la propiedad que se afirma es declarado.

Código y Datos

Repositorio de código: <https://github.com/AndresR08/covert-channel-containment-testbed>
Todas las plantillas de infraestructura, los scripts de agente y el paquete completo de evidencia antes/después (logs de contenedor, datos de tiempo y costo, etiquetas de recursos y las dos

transcripciones de pruebas de credenciales) están en el repositorio bajo evidence/. No contiene ningún exploit: únicamente un token benigno.

Referencias

- [1] Larcher, H., Carreira, A., et al. (2026). Agent Intrusion: Technical Timeline. Hugging Face Blog, 27 July 2026. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- [2] Greenblatt, R., Cotra, A., Wijk, H. (2026). OpenAI–Hugging Face Incident Investigation. METR, 26 August 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>
- [3] Lampson, B. W. (1973). A Note on the Confinement Problem. Communications of the ACM, 16(10), 613–615.
- [4] Ristenpart, T., Tromer, E., Shacham, H., Savage, S. (2009). Hey, You, Get Off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds. Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS '09).
- [5] UK Parliament (2026). Artificial Superintelligence Bill. Bill 4288, introduced 8 September 2026. <https://bills.parliament.uk/bills/4288>
- [6] European Commission (2026). AI Act: Reporting Template for Serious Incidents Involving General-Purpose AI Models with Systemic Risk (Article 55(1)(c)); guidance applying from 2 August 2026. <https://digital-strategy.ec.europa.eu/en/library/ai-act-commission-publishes-reporting-template-serious-incidents-involving-general-purpose-ai>
- [7] State of California (2025). Transparency in Frontier Artificial Intelligence Act (SB 53), critical safety incident reporting to the Governor's Office of Emergency Services. https://en.wikipedia.org/wiki/Transparency_in_Frontier_Artificial_Intelligence_Act
- [8] Congreso de Colombia (2026). Proyecto de Ley 025 de 2026: por medio de la cual se regula la inteligencia artificial en Colombia. Cámara de Representantes. <https://www.camara.gov.co/wp-content/uploads/2026/07/proyectos-ley/documentos/proyecto-36127/P.L.025-2026SC-INTELIGENCIA-ARTIFICIAL.pdf>

Declaración de Uso de LLM

Se utilizó Claude (Anthropic) a lo largo de este proyecto: para estructurar e iterar la infraestructura de Azure (plantillas Bicep, scripts de contenedor), para diseñar e implementar las variantes del control de contención, para redactar y reestructurar este informe, y para verificar fuentes externas mediante búsqueda web antes de citarlas. Toda la infraestructura fue desplegada y todos los resultados experimentales (logs de antes/después, pruebas de credenciales) fueron observados de forma independiente por el autor contra una suscripción de Azure en vivo. Claude no generó ni simuló ninguna evidencia. El autor revisó, probó y aprobó todas las decisiones de diseño, incluida la elección entre variantes de contención y las afirmaciones finales hechas en este informe.